

EFFICIENTNET

By NVIDIA

EfficientNets are a family of image classification models, which achieve state-of-the-art accuracy, being an order-of-magnitude smaller and faster. Trained with mixed precision using Tensor Cores.

[View on Github](#)

[Open on Google Colab](#)

[Open Model Demo](#)



Model Description

EfficientNet is an image classification model family. It was first described in [EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks](#). This notebook allows you to load and test the EfficientNet-B0, EfficientNet-B4, EfficientNet-WideSE-B0 and, EfficientNet-WideSE-B4 models.

EfficientNet-WideSE models use Squeeze-and-Excitation layers wider than original EfficientNet models, the width of SE module is proportional to the width of Depthwise Separable Convolutions instead of block width.

WideSE models are slightly more accurate than original models.

This model is trained with mixed precision using Tensor Cores on Volta and the NVIDIA Ampere GPU architectures. Therefore, researchers can get results over 2x faster than training without Tensor Cores, while experiencing the benefits of mixed precision training. This model is tested against each NGC monthly container release to ensure consistent accuracy and performance over time.

We use [NHWC data layout](#) when training using Mixed Precision.

Example

In the example below we will use the pretrained *EfficientNet* model to perform inference on image and present the result.

To run the example you need some extra python packages installed. These are needed for preprocessing images and visualization.

```
!pip install validators matplotlib
```

```
import torch
from PIL import Image
import torchvision.transforms as transforms
import numpy as np
import json
import requests
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings('ignore')
%matplotlib inline

device = torch.device("cuda") if torch.cuda.is_available() else torch.device("cpu")
print(f'Using {device} for inference')
```

Load the model pretrained on ImageNet dataset.

You can choose among the following models:

TorchHub endpoint	Description
nvidia_efficientnet_b0	baseline EfficientNet
nvidia_efficientnet_b4	scaled EfficientNet
nvidia_efficientnet_widese_b0	model with Squeeze-and-Excitation layers wider than baseline EfficientNet model
nvidia_efficientnet_widese_b4	model with Squeeze-and-Excitation layers wider than scaled EfficientNet model

There are also quantized version of the models, but they require nvidia container. See [quantized models](#)

```
efficientnet.eval().to(device)
```

Prepare sample input data.

```
uris = [
    'http://images.cocodataset.org/test-stuff2017/000000024309.jpg',
    'http://images.cocodataset.org/test-stuff2017/000000028117.jpg',
    'http://images.cocodataset.org/test-stuff2017/000000006149.jpg',
    'http://images.cocodataset.org/test-stuff2017/000000004954.jpg',
]

batch = torch.cat(
    [utils.prepare_input_from_uri(uri) for uri in uris]
).to(device)
```

Run inference. Use `pick_n_best(predictions=output, n=topN)` helper function to pick N most probable hypotheses according to the model.

```
with torch.no_grad():
    output = torch.nn.functional.softmax(efficientnet(batch), dim=1)

results = utils.pick_n_best(predictions=output, n=5)
```

Display the result.

```
for uri, result in zip(uris, results):
    img = Image.open(requests.get(uri, stream=True).raw)
    img.thumbnail((256,256), Image.ANTIALIAS)
    plt.imshow(img)
    plt.show()
    print(result)
```

Details

For detailed information on model input and output, training recipes, inference and performance visit: [github](#) and/or [NGC](#)

References

- [EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks](#)
- [model on NGC](#)
- [model on github](#)
- [pretrained model on NGC \(efficientnet-b0\)](#)
- [pretrained model on NGC \(efficientnet-b4\)](#)
- [pretrained model on NGC \(efficientnet-widese-b0\)](#)
- [pretrained model on NGC \(efficientnet-widese-b4\)](#)
- [pretrained, quantized model on NGC \(efficientnet-widese-b0\)](#)
- [pretrained, quantized model on NGC \(efficientnet-widese-b4\)](#)

Not Working?

Docs

Access comprehensive developer documentation for PyTorch

[View Docs](#)

Tutorials

Get in-depth tutorials for beginners and advanced developers

[View Tutorials](#)

Resources

Find development resources and get your questions answered

[View Resources](#)

Features

Ecosystem

Blog

Contributing

Security

Docs

Discuss

GitHub Issues

Brand Guidelines

Stay up to date

Facebook

Twitter

YouTube

LinkedIn

Mastodon

PyTorch Podcasts

Spotify

Apple

Google

Amazon

Terms | Privacy

© Copyright The Linux Foundation. The PyTorch Foundation is a project of The Linux Foundation. For web site terms of use, trademark policy and other policies applicable to The PyTorch Foundation please see www.linuxfoundation.org/policies/. The PyTorch Foundation supports the PyTorch open source project, which has been established as PyTorch Project a Series of LF Projects, LLC. For policies applicable to the PyTorch Project a Series of LF Projects, LLC, please see www.lfprojects.org/policies/.